

Kanon M0: Cost Baseline Report

RedStone Oracle Adaptor, in-zkVM verification cost

v1.1

2026-08-11

Equilibrium Group Oy
Meritullinkatu 1
Helsinki
Finland

Logos
RFP-020

TABLE OF CONTENTS

Scope 3

1. The answer in one page 4

2. What was measured, and why in two places 6

 2.1 Method 6

 2.2 Two execution modes, two different costs 6

 2.3 The message size that matters: 77 bytes 7

3. Results 8

 3.1 The two accelerators are different in kind 8

 3.2 Recovery does not amortise 9

 3.3 The LEZ framework is cheap, but not free 9

 3.4 Reproducibility 10

4. In-program versus precompile: first delta sketch 11

5. Recommendations 12

 5.1 In-program path: accelerate secp256k1, leave keccak256 in software 12

 5.2 Heartbeat width: keep the RFP defaults, revisit when fees land 12

 5.3 Push versus pull: default to push, price pull rather than discourage it 12

 5.4 Precompile follow-on: recommend it, for recovery only 13

6. Caveats 14

7. What we need from Logos 15

 7.1 The operating envelope 15

 7.2 Which LEZ version the project targets 15

 This is a solved problem elsewhere in the Logos estate 15

 What we propose 16

 The two asks 16

8. Reproducing this 18

Author: Equilibrium Group Oy
Status: Milestone deliverable
Created: 2026-08-07
Label: zkvm-cost-baseline
Client: Logos
Target: RFP-020 milestone M0, task M0-12

Scope

RFP-020 makes the in-zkVM cost of RedStone signature verification the primary M0 deliverable, and asks that the operating envelope be agreed with Logos before build decisions are made. This report is the basis for that conversation. It states what was measured, how, what it costs, what we recommend, and the two decisions we need from Logos.

Full method detail, the complete tables and the reproduction instructions are in `m0/cost-baseline/README.md` and `m0/lez-probe/README.md`. Pinned versions are in `m0/versions.md`. Every figure quoted here is asserted by a test that runs in CI on x86_64 and ARM64, so this document cannot quietly drift from the build.

Revision v1.1 (2026-08-11). The decomposition in section 1 now separates the LEZ framework's per-package instruction handling from the cost of recovery, measured with a third guest program rather than inferred by subtracting the empty-program floor. That work survives a cryptographic precompile, so the precompile sketch in section 4 is revised from a 15x–20x reduction to 8x–9x. Every other figure is unchanged, as are all recommendations and both decisions requested of Logos.

1. The answer in one page

A 3-of-N single-feed price update costs 1,906,737 cycles inside a LEZ program: 5.68% of LEZ's 32M per-transaction budget. It fits comfortably, roughly 17 updates to the budget, and the shape of the cost is unusual enough to drive several design decisions.

	cycles	share of the 32M budget
LEZ framework floor	39,103	0.12%
1 signer	664,117	1.98%
3 signers (RFP default)	1,906,737	5.68%
5 signers	3,150,337	9.39%

Recovery is the overwhelming majority of the cost. Decomposing that 3-of-N update:

component	cycles	share
LEZ framework floor	39,103	2.05%
LEZ instruction handling, 3 packages	119,187	6.25%
3 × keccak256 over 77 bytes, software	51,956	2.72%
3 × secp256k1 recovery and signature parse	1,696,491	88.97%
total	1,906,737	100%

The framework appears on two rows because part of its work scales with the instruction and part does not. The floor is what an empty LEZ program costs; the second row is the framework deserializing the instruction and echoing it back out, which grows with the signer count and is measured separately rather than left to fall into the recovery row. Shares are rounded and do not sum to exactly 100%.

Four consequences, each expanded below:

- 1. Use RISC Zero's secp256k1 accelerator; decline its keccak256 accelerator.** The first is a 19.7x cycle reduction with no side effects. The second lowers cycles while costing 26.35 s of proving and 223 KB of proof, because it moves work to a coprocessor the cycle count cannot see.
- 2. Threshold width is linearly priced and there is no engineering remedy.** Per-recovery cost is flat across 1, 3 and 5 signers, so nothing is shared between recoveries. Moving 3-of-N to 5-of-N costs 65% more, permanently.

3. **The RFP's heartbeat defaults are affordable.** At 5.68% per update there is no cost argument for widening intervals.
4. **A recovery precompile is worth asking Logos for; a keccak256 one is not.** A precompile addresses a measured 88.97% of the program, an 8x to 9x reduction. A keccak precompile would address 2.72%.

2. What was measured, and why in two places

Verifying a RedStone update on-chain requires two primitives that Ethereum provides natively and LEZ does not: **keccak256** over each data package’s signed bytes, and **secp256k1 ECDSA public-key recovery**, once per signer. On LEZ both run as RISC-V instructions in a zkVM, so they cost whatever their software implementation costs. That cost is the deliverable.

Two independent measurements were built, deliberately:

- `m0/cost-baseline/` measures the two primitives on **bare RISC Zero**, with no dependency on LEZ, SPEL, a sequencer or a live RedStone feed. It isolates the primitives from anything that would add noise, and it is where the accelerator comparison is made.
- `m0/lez-probe/` measures the same workload **inside a real LEZ program**, via `lee_core`, driving `Program::write_inputs` under LEZ’s own 32M `session_limit`. This is what makes the figures directly comparable to the budget, and what separates public execution from private.

The headline figure comes from the second. The first exists because a cross-configuration comparison needs the primitives measured without a framework around them.

2.1 Method

Every workload runs **twice against byte-identical input**, once with the cryptographic work enabled and once with it skipped; the difference is the cost of the primitive, with zkVM startup, input deserialization and journal commit cancelled out. Inside the LEZ program the same subtraction needs a **third** guest, identical to the verifying one but with the cryptography removed, because a LEZ program has no second run to cancel against and part of the framework’s own input handling scales with the signer count. The three guest configurations differ **only** in their `[patch.crates-io]` section, so guest source is byte-identical across them and the only variable is which implementation the linker chose. Recovery success is asserted in the guest, so a run that took a cheap error path cannot be misread as a measurement.

2.2 Two execution modes, two different costs

This distinction matters for every number below, and it is a LEZ property rather than a RISC Zero one:

mode	proven	cycle cap	binding cost
public	no	32M	cycles
private	yes	none	proving time and proof size

Public execution is capped in cycles and **not proven**, so cycles are the operative cost and are directly comparable to the budget with no conversion. Private execution proves the same program with no session limit, so there proving time and proof size are what a consumer pays. We report both, and say which applies where.

2.3 The message size that matters: 77 bytes

Taken from RedStone's own Rust SDK wire format, the signed range of one data package carrying a single 32-byte data point is $1 * (32 + 32) + 4 + 6 + 3 = \mathbf{77 \text{ bytes}}$. Each signer signs its own 77 bytes, so an M-of-N update needs **M distinct hashes**, not one hash checked against M signatures. 77 bytes fits in a single keccak256 permutation, the cheapest case the primitive has, so **there is nothing to win by shrinking the payload**.

3. Results

3.1 The two accelerators are different in kind

The full 3-of-N update on bare RISC Zero, all three configurations. Proving figures are the best of 3 runs with the observed spread, because wall-clock noise can otherwise invert a comparison:

configuration	cycles	prove time	proof size	segments
software (neither accelerator)	33,347,017	not run	not run	34
accelerated (both)	1,769,222	164.02 s	785,897 B	2
mixed (secp256k1 only)	1,813,434	137.67 s	562,764 B	2

Choosing mixed over accelerated is worth 26.35 s and 223 KB per update, for 2.5% more cycles. The software configuration is ruled out on cycles alone at 19x the others. Receipts are composite with 0 assumptions in all three, so the choice does not change what an on-chain verifier must handle.

What the three hashes actually cost, per configuration, as a difference within each:

configuration	3 hashes, cycles	added prove time	added proof bytes
accelerated	7,744	+26.90 s	+223,133
mixed	51,956	-1.53 s (below noise)	0

The **0** is exact, not rounded, and follows from proof size being quantised by segment: the extra 51,956 cycles stay inside the same 2 segments, so the proof is byte-identical. Proof size is predictable from cycles at roughly **281 KB per segment**, which means a cycle budget is also a proof-size budget.

The accelerated row is the contrast that matters: +223,133 bytes while still using 2 segments. That is not extra segments, it is a **separate coprocessor proof attached alongside them**, and its cost is paid per proof rather than per hash. Measuring at two hash counts locates the crossover: at the 77-byte RedStone size the two paths are **level on proving time at about 16 hashes**, which is 5 to 6 feeds at a 3-of-N threshold. Below that software keccak is cheaper; above it the accelerator pulls ahead. Proof size crosses much later, around 60 hashes.

For the deliverable as scoped, none of this is close. A single-feed 3-of-N update is three hashes, where software keccak is free on both axes and the accelerator costs 26.90 s and 223 KB.

3.2 Recovery does not amortise

Per primitive, on bare RISC Zero:

signers	software total	software per recovery	accelerated total	accelerated per recovery	speed-up
1	11,074,701	11,074,701	565,551	565,551	19.6x
3	33,228,606	11,076,202	1,686,893	562,298	19.7x
5	55,375,167	11,075,033	2,817,096	563,419	19.7x

Total cost grows **linearly** and per-recovery cost is **constant**: each recovery is an independent scalar multiplication with nothing shared. The absence of the alternative is the finding. If recoveries could share work, per-recovery cost would fall as the count rose. It does not, so **there is no amortisation to exploit and widening the threshold is permanently linear**. Inside the LEZ program the same property gives about **621,555 cycles per additional signer**.

keccak256, for completeness, at the sizes that matter (net of measured loop bookkeeping of 94 cycles per iteration):

message bytes	software	accelerated	speed-up
32	17,182	2,118	8.1x
77 (RedStone)	17,498	2,433	7.2x
141	34,416	3,977	8.7x

Note the step rather than a slope: 77 costs about what 32 costs, and 141 costs double, because keccak256 absorbs in 136-byte blocks.

3.3 The LEZ framework is cheap, but not free

Set against the bare-RISC-Zero figures for the same configuration, the LEZ program framework adds little: proving time is identical within noise and the proof grows by 1,873 bytes, 0.3%. In cycles it has two parts. The floor — an empty LEZ program — is 39,103 cycles, 0.12% of the budget. On top of that the framework deserializes the instruction and echoes it back out, which **scales with the signer count**: 119,187 cycles at 3 signers, about 39,659 per package. Together that is **158,290 cycles at 3 signers, 0.47% of the budget and 8.3% of the program**.

That second part is measured with a third guest program, identical to the verifying one but with the cryptography removed, so the framework's per-package work is subtracted rather than left inside the recovery figure. Subtracting only the floor would have put the cryptography at 1,867,634 cycles; measured against the third program it is 1,748,447, which cross-checks to

within 0.55% against the bare-RISC-Zero figures for the same three recoveries and hashes (1,686,893 + 51,956 = 1,738,849).

signers	framework floor	instruction handling	cryptography	total
1	39,103	39,869	585,145	664,117
3	39,103	119,187	1,748,447	1,906,737
5	39,103	198,505	2,912,729	3,150,337

Private mode, proving the same program:

measurement	cycles	prove time	proof size
LEZ framework floor	39,103	8.57 s	244,430 B
3 signers	1,906,737	137.45 s	564,637 B

Worth noting on its own: any LEZ program costs at least 8.57 s and 244 KB to prove, against 2.23 s and 209 KB for an empty RISC Zero guest. Verification is still the overwhelming majority of a private-mode transaction.

3.4 Reproducibility

The 3-of-N workload was built from scratch and run on **three machines across two instruction sets**. Every deterministic quantity matched **byte for byte**: cycles, segments, proof size.

3-of-N, mixed	NixOS x86_64	Ubuntu x86_64	macOS ARM64
cycles, hash + recover	1,813,434	1,813,434	1,813,434
segments	2	2	2
proof size	562,764 B	562,764 B	562,764 B

Two conditions are jointly necessary: the six lockfiles pin crate resolution, and rzup pins the guest toolchain. One machine initially had r0vm 3.0.3 with guest rustc 1.88.0 and had to be brought to parity before its numbers meant anything. The **host** compiler is not a condition: 1.93.0 and 1.97.1 gave identical counts, because the host builds only the driver.

Every published figure is now asserted by exact equality in CI, on both architectures, so this is a standing check rather than a one-off observation.

4. In-program versus precompile: first delta sketch

RFP-020 asks M0 for a first sketch; the full per-mode report is M3.

A recovery precompile would address the **measured 88.97%** of the program. What remains is the framework — both its floor and its per-package instruction handling, neither of which a cryptographic precompile touches — the hashing, and whatever guest-side marshalling a precompile syscall costs. Taking that last residual at 1,000 to 10,000 cycles per call:

residual assumption	program total	share of budget	reduction
1,000 cycles per call	213,246	0.64%	8.9x
10,000 cycles per call	240,246	0.72%	7.9x

So the sketch is an **8x to 9x reduction**, taking budget use from 5.68% to under 0.8%. Only the syscall cost is an assumption; no such precompile exists to measure. The framework’s 158,290 cycles are measured, and together with the hashing they cap the reduction at about **9.1x** however cheap the syscall turns out to be.

Why the ratio is so unlike EVM. RFP-020 cites RedStone’s EVM path at 50K to 100K gas end to end, where a native `ecrecover` is about 3,000 gas, so three recoveries are roughly 9,000 gas, some 9% to 18% of the total. In LEZ without a native primitive the same three recoveries are **89%** of the program. The missing primitive does not make verification somewhat dearer; it moves recovery from a minor line item to essentially the whole cost. That is also why pairing a keccak256 precompile with it is not worth it, at 2.72% of the program.

5. Recommendations

5.1 In-program path: accelerate secp256k1, leave keccak256 in software

The `mixed` configuration. It is 2.5% more cycles than accelerating both, and saves 26.35 s of proving and 223 KB of proof per update. It is the cheaper choice on every axis that binds at the scope RFP-020 describes.

Revisit if a single program ever verifies **more than about 16 hashes**, roughly 5 to 6 feeds at 3-of-N, at which point the keccak coprocessor's fixed cost starts to amortise. In the band between roughly 16 and 60 hashes the accelerator is faster while still producing the larger proof, and which wins there depends on whether proving time or proof size is the binding cost. That is a LEZ question rather than a RISC Zero one.

5.2 Heartbeat width: keep the RFP defaults, revisit when fees land

At 5.68% of the budget per update there is no cost argument for widening intervals. Note that LEZ's 32M constant is marked `TOD0`: Make this variable when fees are implemented upstream, so the denominator should be expected to change, and the decision should be revisited when it does.

5.3 Push versus pull: default to push, price pull rather than discourage it

Both modes run the same verifier over the same payload, so a pull read costs the same 1.9M cycles as a push update. What differs is how often that cost is paid: push pays once per update and every reader shares the result, while pull pays in full on every read, and carries 440 payload bytes each time. The multi-feed batching soft requirement amortises across feeds within a single update, which benefits push only. Push should therefore be the recommended integration.

But pull is comfortably affordable, not prohibitive. A pull consumer spends 5.68% of its own transaction budget on verification and keeps roughly 94% for its own logic. The argument for push is fee amortisation, not budget pressure, so the consumer documentation should put the per-read figure in front of the decision rather than steer consumers away from a mode the RFP requires.

5.4 Precompile follow-on: recommend it, for recovery only

On cost grounds, and only for recovery. It addresses a measured 88.97% for an 8x to 9x reduction. A keccak256 precompile addresses 2.72% and is not worth the interface.

6. Caveats

Stated plainly, because several of them bound how far these numbers should be carried.

- **Not the full transaction.** This measures program execution. Payload decode, the median across signers, the canonical RFP-019 account write and admin checks are M1 and M2 work, and they add to the 5.68%.
- **The precompile residual is only partly assumed.** The 88.97% a precompile would address is measured, and so is the 8.3% of framework cost that survives it; the 1,000 to 10,000 cycles per call of syscall marshalling is an estimate.
- **Stage two of private execution is not measured.** The wrapper circuit's cost is additional. It is LEZ's own and roughly fixed with respect to the adaptor, but a full private-mode transaction cost would include it.
- **Proving time is hardware-dependent and noisy.** Reported as the best of several samples with the spread shown. Cycles and proof sizes are deterministic; proving wall-clock is not, and single runs have been observed to order two configurations wrongly.
- **Upstream is early and moving.** LEZ's 32M cap is marked as provisional upstream. These figures are built against `lee_core` v0.2.1 and upstream has since tagged v0.2.4, so the pin is already behind. See the decision in section 7.2.
- **GPU proving was measured but is not the basis of any recommendation here.** It changes proving wall-clock, not cycles, and cycles are what public execution charges.

7. What we need from Logos

Two decisions. The first is the one RFP-020 anticipated; the second emerged from standing up the toolchain and is the more urgent of the two.

7.1 The operating envelope

Confirm, amend or reject the four recommendations in section 5: the in-program path, the heartbeat defaults, the push-versus-pull emphasis, and whether to pursue a recovery precompile as a follow-on. Section 5.1 is the one that constrains M1’s implementation, so it is the one we most need settled.

7.2 Which LEZ version the project targets

There are two LEZ versions in play, **784 commits and three months apart**:

	commit	tag	date
What these cost figures are built against (<code>lee_core</code>)	15144ddb	v0.2.1	2026-08-02
What the <code>lgs</code> toolchain pins for its sequencer	cf3639d8	v0.1.2	2026-04-27

Upstream has since tagged v0.2.2, v0.2.3 and v0.2.4, so **neither pin is current**. We tested the override rather than assuming: `lgs` **builds** a v0.2.1 sequencer in 2m02s, so this is not a deep API break, but `lgs test-node start` **fails** against it. The reason is specific, and it bounds the work more narrowly than it first appears:

LEZ v0.2.x replaced the sequencer’s genesis model. `genesis_id`, `is_genesis_random`, `initial_accounts` and `initial_commitments` are gone, replaced by a single tagged `genesis` array of `supply_bridge_account` and `supply_account` entries, plus a new `bedrock_config.funding_key`. Scaffold requires a numeric `genesis_id` (`src/testnode/mod.rs`) and seeds state by writing `initial_public_accounts / initial_private_accounts`, neither of which exists in v0.2.x. The crate tree also moved under `lez/`.

The blocked surface is narrower than that list suggests: only the isolated `test-node` harness reads `genesis_id`. The long-lived `localnet` sequencer discards it, so running a v0.2.x sequencer is not itself blocked by this change.

This is a solved problem elsewhere in the Logos estate

The important finding is that we are not first, and the path is public:

- **logos-co/eth-lez-atomic-swaps already runs scaffold against LEZ v0.2.2** (d6e4ae69), one tag *newer* than our figures, pushed as recently as today. It pins the LEZ checkout via

[`repos.lez`].path (a caller-provided checkout, which scaffold validates but never resets) and carries a documented bridge script, `scripts/scaffold-setup.sh`, for the wallet gaps.

- **A v0.2.x-aligned SPEL exists.** Scaffold's default SPEL (73fc462e) vendors LEZ v0.1.2; the SPEL that project pins (3d639076) vendors v0.2.0-rc3. So the SPEL-and-LEZ alignment constraint is satisfiable on the v0.2.x line, not only on v0.1.2.
- **Upstream is mid-migration.** Scaffold issue #240 and PR #246 ("support the LEZ v0.2.0 wallet across setup/run/doctor", 324 additions across 9 files) have been open since 2026-07-30 and updated 2026-08-05. That PR removes the need for the bridge script for `setup`, `run` and `doctor`.

What we propose

Nothing changes for M0, and no measurement needs redoing. No M0 figure depends on a sequencer: the cost of a program is a property of its guest ELF, and nothing in either measurement harness runs one. We raise it now because now was the cheap moment to find it.

M1-05 is the forcing function. It asks for integration tests that transact against a standalone sequencer, and the moment a test submits a transaction, that sequencer must speak the protocol our guest was built against. Alignment is therefore a prerequisite for writing M1-05, not a task that follows it, and M2-19's end-to-end verify-and-publish depends on the same thing. Our recommendation:

1. **Adopt the v0.2.x triple that is already proven downstream:** LEZ v0.2.2 or later via [`repos.lez`].path, the SPEL that vendors v0.2.x, and the bridge script until PR #246 lands. None of that requires a change to a repository we do not own.
2. **Settle the pin before M1 writes verifier-core.** Moving off v0.2.1 later means re-measuring every figure here and reissuing this report, and possibly API churn in M1 code rather than only new numbers. Once, early, is the cheap order.
3. **File the one gap PR #246 does not cover.** It fixes `setup`, `run` and `doctor`; the numeric `genesis_id` requirement is separate and belongs to `test-node`, which is what isolated per-test nodes would use. `localnet` is unaffected, so an M1-05 built on one long-lived sequencer may not need it at all.

The two asks

Narrower than it first appeared, and neither item blocks M0 or M1:

- **The intended target pin.** If v0.2.2 is what the rest of the estate is standardising on, we will move to it rather than staying on v0.2.1 alone.
- **Agreement on one scope change.** M0-01 as written asks for the `lgs` toolchain and a standalone sequencer to be stood up in M0. No M0 figure depends on either, and a spike confirmed that while settling the version question above, so we propose the

toolchain move to M1, where M1-05 first needs it. M0's other pinning obligations, the LEZ, RISC0 and r0vm versions the measurements depend on, are met in `m0/versions.md`.

- **The expected timeline for scaffold #240 / #246**, and whether `test-node v0.2.x` support is in scope for it. That determines whether we carry the bridge script or wait.

8. Reproducing this

```
nix-shell                                # NixOS; else rustup + rzup
cargo test --release --locked --workspace # every figure above, seconds
cargo test --release --locked --workspace -- --ignored # proving figures, minutes
cargo run --release -p lez-probe -- --prove --repeat 2 # the headline number
```

Guardrails and CI: `.github/workflows/guardrails.yml` runs the cycle assertions on x86_64 and ARM64, plus `cargo fmt` and `cargo clippy -D warnings`. The proving assertions are `#[ignore]`d because they take minutes and are hardware-dependent; they are run by hand, and were last confirmed green on 2026-08-07.

document	contents
m0/cost-baseline/README.md	bare-RISC-Zero method, full tables, accelerator analysis
m0/lez-probe/README.md	LEZ program method, budget figures, precompile sketch
m0/versions.md	every pinned version, the two LEZ versions, toolchain setup
m0/cost-baseline/host/tests/guardrails.rs	the asserted constants for sections 3.1 to 3.2
m0/lez-probe/host/tests/guardrails.rs	the asserted constants for section 3.3

A guardrail failure is a prompt to re-measure and re-tag deliberately, not necessarily a defect.